



AstroSpot

Planificateur de soirées d'observation des étoiles — une architecture micro-services qui croise pollution lumineuse et météo pour noter la qualité du ciel.

BINÔME

Dayssam Bakaar

Kais Hammache

CLASSE

LSI 1 · APP

Ingé 2

STACK

TypeScript · Node 22

Express · Jest · Docker

ÉCHÉANCE

21 juin

Remise sur Moodle

01 Présentation & cas d'usage

AstroSpot répond à une question concrète d'astronomie amateur : « *Vaut-il la peine de sortir le télescope ce soir, à cet endroit ?* »

L'application combine deux facteurs déterminants pour observer les étoiles :

- la **pollution lumineuse** du site (échelle de Bortle : 1 = ciel pur → 9 = ciel urbain) ;
- la **couverture nuageuse** prévue, récupérée auprès d'une **API météo externe** (Open-Meteo).

Elle en déduit un **score de qualité du ciel** (0–100), une **note** (EXCELLENT / GOOD / FAIR / POOR) et une **recommandation**.

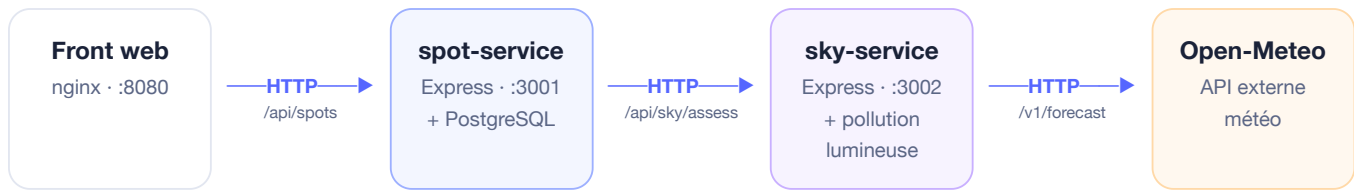
Pourquoi ce sujet pour un projet DevOps ? Le découpage fonctionnel impose **deux services back** qui **communiquent par HTTP**, créant **deux frontières réseau** (`spot` → `sky` et `sky` → `météo`). Cela rend les **mocks web** exigés par le sujet réellement nécessaires — et non artificiels.

```
# Calcul du score de qualité du ciel
lightScore = (9 - bortleClass) / 8 × 100    # Bortle 1 → 100, Bortle 9 → 0
cloudScore = 100 - couvertureNuageuse(%)
score      = round(0.5 × lightScore + 0.5 × cloudScore)    # borné [0,100]

rating     = EXCELLENT(≥80) · GOOD(≥60) · FAIR(≥40) · POOR(<40)
recommended = score ≥ 60
```

02 Architecture logicielle

2.1 — Vue micro-services



Deux services back indépendants, conteneurisés et orchestrés par `docker-compose`. `spot-service` porte le domaine (spots & planification); `sky-service` est un service de calcul sans état, réutilisable.

2.2 — Architecture en couches (par service)

spot-service	sky-service
Controller · <code>spotController.ts</code>	Controller · <code>skyController.ts</code>
Services · <code>spotService.ts</code> , <code>skyClient.ts</code>	Services · <code>skyConditionsService.ts</code> , <code>weatherClient.ts</code>
Data · <code>SpotRepository</code> (InMemory · PostgreSQL)	Data · <code>LightPollutionRepository</code>

Règle de dépendance. Chaque couche ne dépend que de la couche inférieure, toujours **via une interface** (`SpotRepository`, `WeatherClient`, `SkyGateway`). On peut donc remplacer l'implémentation en mémoire par PostgreSQL sans toucher au reste — et tester chaque couche isolément.

2.3 — Séquence : planifier une observation

```
Front → spot-service : POST /api/spots/{id}/plan { date }
spot-service → repository : get(spot)
spot-service → sky-service : GET /api/sky/assess?lat&lon&date
sky-service → Data : findNearest(bortle)
sky-service → Open-Meteo : GET /v1/forecast (cloudcover)
Open-Meteo → sky-service : couverture nuageuse
sky-service → spot-service : { score, rating, recommended }
spot-service → Front : { spot, date, assessment, recommended }
```

03 Tests, couverture & qualité

56

TESTS VERTS

100%

LIGNES · SKY

98%

LIGNES · SPOT

4

COUCHES TESTÉES

Couverture de code

SERVICE	TESTS	STATEMENTS	BRANCHES	FUNCTIONS	LIGNES
sky-service	30	100 %	92.59 %	100 %	100 %
spot-service	26	98.07 %	94.11 %	100 %	98.07 %

Couverture lignes globale (143 / 147 lignes exécutables)

Des seuils minimaux sont imposés dans `jest.config.js` (`coverageThreshold` : 80 % branches / 85 % lignes). La CI échoue si la couverture passe sous ces seuils — la qualité est donc verrouillée automatiquement.

Stratégie de test par couche

COUCHE	TYPE DE TEST	OUTILS	EXEMPLE
Data	Unitaire	Jest	<code>lightPollutionRepository.test.ts</code>
Services	Unitaire + mocks	Jest, <code>nock</code>	<code>skyConditionsService.test.ts</code>
Controller	Intégration HTTP	Supertest, <code>nock</code>	<code>spotController.test.ts</code>
Inter-service	Mock web	<code>nock</code>	<code>skyClient.test.ts</code>

Mocks web

Deux frontières HTTP simulées avec `nock` :

- `sky` → `Open-Meteo` (API externe)
- `spot` → `sky` (appel inter-service)

→ tests isolés et déterministes, sans dépendre du réseau.

Qualité logicielle

- TypeScript `strict` (+ `noUnused*`)
- ESLint 0 erreur
- Prettier + `.editorconfig`
- Erreurs typées → codes HTTP
- Découplage par interfaces

04 Conteneurisation & intégration continue

Docker (≥ 2 services back)

`docker-compose.yml` orchestre 4 conteneurs : `spot-service` , `sky-service` , `postgres` et `frontend` . Chaque service back a un `Dockerfile` **multi-stage** (build TypeScript → image runtime `node:22-alpine`) avec `HEALTHCHECK` .

```
make up # docker compose up --build → front sur http://localhost:8080
```

Pipeline CI — GitHub Actions

ÉTAPE	JOB	ACTION
1	test (matrice x2)	<code>npm ci</code> → <code>lint</code> → <code>build</code> → <code>test:coverage</code>
2	test	publication des rapports de couverture en artefacts
3	docker	build des 3 images + <code>docker compose config</code>

Pas de **Continuous Delivery** (hors programme) : le pipeline s'arrête au build / test / qualité, conformément au sujet.

Conformité au sujet

EXIGENCE	STATUT	RÉALISATION
Dépôt Git	✓	dépôt + historique de commits
Pipeline CI	✓	GitHub Actions (lint·build·test·docker)
Architecture en couches	✓	Data / Services / Controller stricte
≥ 2 services back + Docker	✓	spot-service + sky-service
Tests de toutes les couches	✓	Jest + Supertest +nock
Bonne couverture	✓	≥ 98 % lignes, seuils imposés
Qualité élevée	✓	TS strict · ESLint · Prettier
Bonus — Front web	✓	frontend nginx
Bonus — Base de données	✓	PostgreSQL 16

05 Google Labs (Google Skills / Qwiklabs)

Parcours et labs Google Cloud réalisés — chaque évaluation à **100 %** :

- *A Tour of Google Cloud Hands-on Labs* — Lab
- *Getting Started with Google Cloud* — Path
- *Create a Virtual Machine* — Lab
- *Build Infrastructure with Terraform on Google Cloud* — Course
- *Infrastructure as Code with Terraform* — Lab

The screenshot shows the Google Skills 'Progress' page. The user is Dayssam BAKAAR, a member since 2026, with a 'Starter' badge. The page displays a list of completed activities:

Activity	Type	Date started	Date finished	Score	Passed
Infrastructure as Code with Terraform	Lab	Apr 23, 2026	Apr 23, 2026	Assessment: 100%	✓
Build Infrastructure with Terraform on Google Cloud	Course	Apr 23, 2026			
Create a Virtual Machine	Lab	Apr 23, 2026	Apr 23, 2026	Assessment: 100%	✓
Getting Started with Google Cloud	Path	Apr 23, 2026			
A Tour of Google Cloud Hands-on Labs	Lab	Apr 23, 2026	Apr 23, 2026	Assessment: 100%	✓

Capture — Google Skills, page « Progress » : labs complétés (Assessment 100 %), compte Dayssam Bakaar.

Limites & perspectives

- Open-Meteo ne couvre que ~16 jours de prévision ; au-delà, `sky-service` renvoie un `502` (dégradation gracieuse **testée**).
- Catalogue de pollution lumineuse volontairement réduit (couche Data en mémoire).
- Pistes : authentification, historique des sessions en base, phase lunaire.